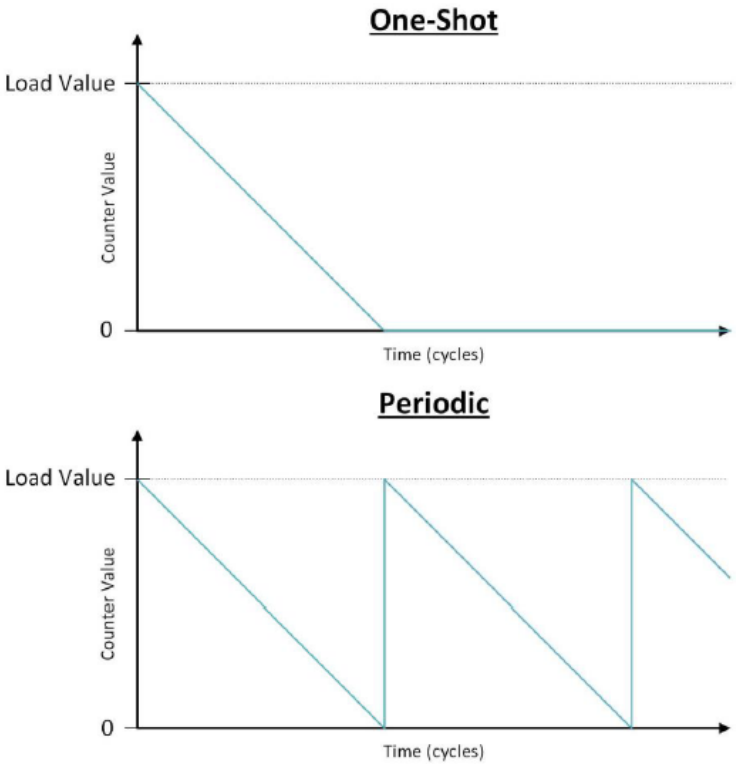
Periodic Timers

ECE 2564, Embedded Systems

Fall 2025

Motivation for Periodic Timers

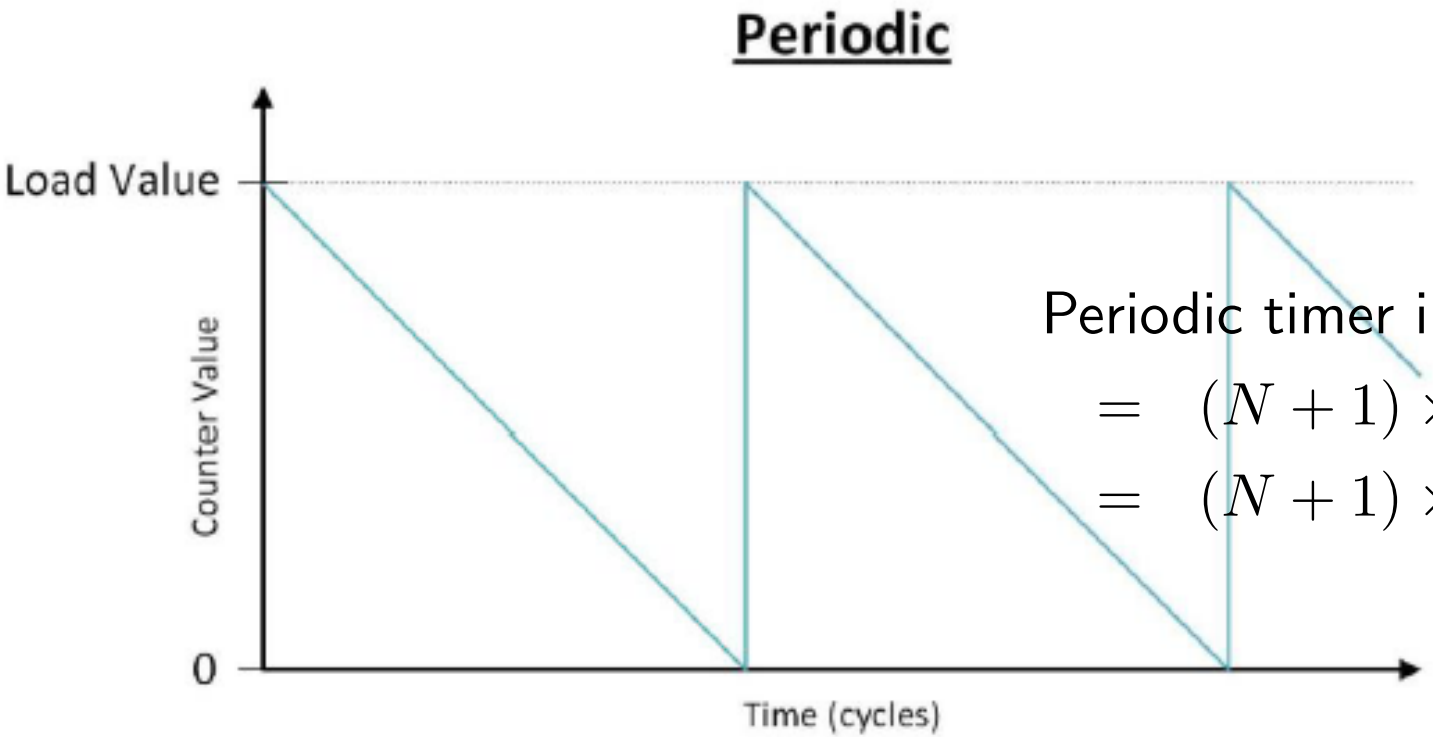
- The time interval that we can measure with a one-shot timer is very limited; periodic timers can measure longer time intervals
- However the number of hardware timers is limited — there aren't enough



One-shot timer interval $= N \times \mathrm{P} \times T_s$

Let the timer roll-over periodically

Periodic Timer Interval



Periodic timer interval

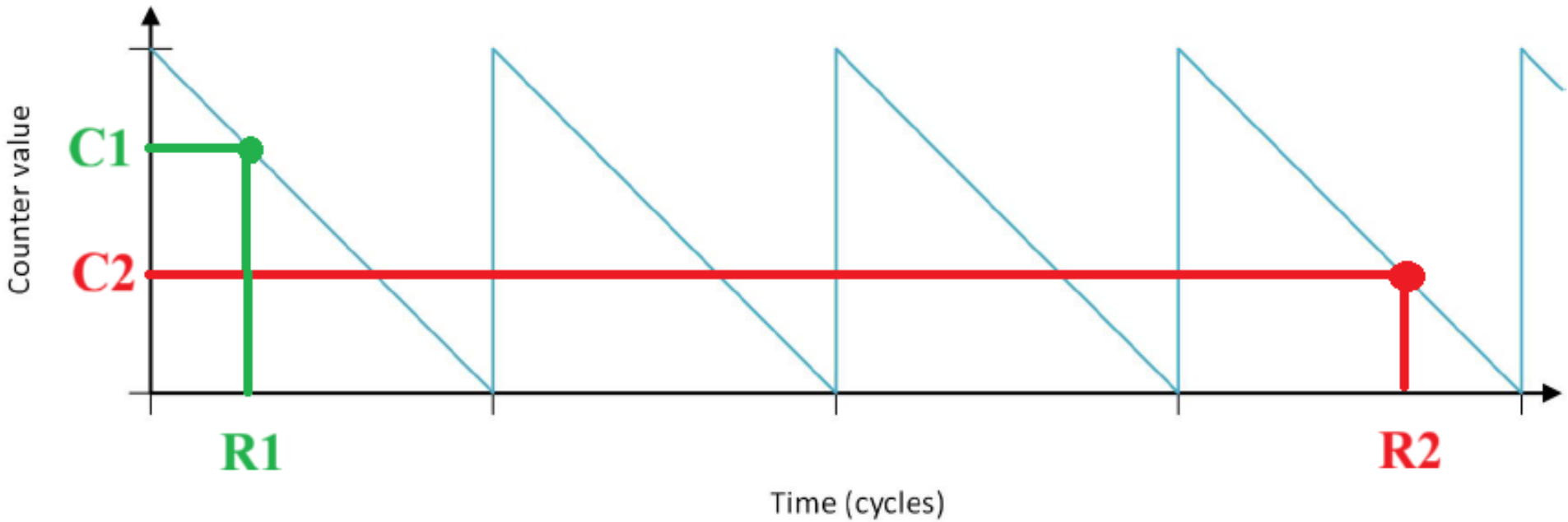
$$= (N+1) \times \mathrm{P} \times T_s$$

$$= (N+1) \times T_C$$

$$N + 1 = (N \text{ cycles for counting}) + (1 \text{ cycle for reloading})$$

Measuring Elapsed Time in Periodic Mode

Let R_i denote the number of rollovers that have occurred up to event i
Let C_i denote the counter value at the time of event i



Measuring Elapsed Time in Periodic Mode

(NOTE: Assume R2 $\geq$ R1)
Event1: (R1, C1)
Event2: (R2, C2)

How many counter cycles have elapsed between Event1 and Event2?

$$\text{Number of elapsed counter cycles} = (\text{C1} - \text{C2}) + (\text{R2} - \text{R1}) \times (N + 1)$$

Multiply the number of elapsed counter cycles by T_C to get the elapsed time between Event2 and Event1

Measuring Elapsed Time in Periodic Mode

Example: $N = 999$
Event1: (1, 100)
Event2: (3, 200)

Event2 and Event1 are separated by $(\text{C1} - \text{C2}) + (\text{R2} - \text{R1}) \times (N + 1)$ cycles:

$$(100 - 200) + (3 - 1) \times 1000 = 1900 \text{ cycles}$$

“Software” Timers

- We are provided with an Application Programming Interface (API) consisting of structs and functions that allow us to build an unlimited number of “software” timers using a single hardware timer
- Analogy: People can use one clock on the wall to measure time for their own use either as a timer or a stopwatch

Key Steps for Implementing Software Timers

1. Set up the MSP432 *system clock* (beyond ECE2564 course)
2. Set up the *wall clock* using a hardware timer
3. Declare a variable that counts the number of *rollovers* from the beginning of timer
4. Create a structure that holds R1 and C1 and the number of cycles the timer needs to wait
5. When the timer is started, set R1 and C1 with the wall clock values
6. Calculate the elapsed time since the software time was started
7. Other functionalities are availble from the software timer

Code Snippets for the "Software" Timer

- We'll see code snippets from `timer.c` and `timer.h` from our `example_GuessTheColor` project
- The key to understanding the following pages:
  - Suppose we are constructing a 2 minute timer
    - We must use the periodic mode
    - How do we determine when the time has expired?

Key Timing Parameters (timer.h)

```
#define SYSTEM_CLOCK        48000000

#define LOADVALUE           0xFFFFFFFF
#define PRESCALER           1
```

Declarations (timer.h)

```
struct _SWTimer
{
    // The number of hardware timer cycles which must elapse before the timer expires
    uint64_t cyclesToWait;

    // The starting counter value of the hardware timer, set when the timer is started
    uint32_t startCounter;

    // The starting rollover value of the hardware timer, set when the timer is started
    uint32_t startRollovers;
};
typedef struct _SWTimer SWTimer;

// Constructs a Software timer. All timers must be constructed before starting them.
SWTimer SWTimer_construct(uint64_t waitTime_ms);

// Starts a software timer. All constructed timers must be started before use.
void SWTimer_start(SWTimer* timer_p);

// A helper function used for determining how many cycles have elapsed since the
// timer was started. You do not need to call this function outside of Timer.c.
uint64_t SWTimer_elapsedCycles(SWTimer* timer_p);

// Determines if the timer has expired - i.e. if enough time has passed since the timer was started
bool SWTimer_expired(SWTimer* timer_p);
```

Declarations (timer.h)

```
// Stopwatch functionality. Determines the number of microseconds since the timer
// was started.  US is for micro seconds
uint64_t SWTimer_elapsedTimeUS(SWTimer* timer_p);

// Determines the percentage of time elapsed before expiration. When the timer
// is started, the value is 0.0, and as time elapses, the value rises to 1.0.
double SWTimer_percentElapsed(SWTimer* timer_p);

// Initializes the global clock system for the MSP432, as well as a hardware
// timer under which all of the software timers are based.
void InitSystemTiming();
```

Step 1: Setup System Clock – DO NOT MODIFY THIS

```
void InitSystemTiming()     // from timer.c
{
    // Before initializing anything else, disable all interrupts
    Interrupt_disableMaster();

    // Before changing the clock frequency, we need to change the flash control to use 2 wait
    // states (2 delayed cycles per flash read). IF YOU DO NOT CHANGE YOUR FLASH CONTROL BEFORE
    // CALLING CS_setDCOFrequency(), YOU WILL BRICK YOUR BOARD AND WILL NEED TO PERFORM A
    // FACTORY RESET.
    //
    // The reason why we need to change the Flash Control settings is because although calling
    // CS_setDCOFrequency() may change the system clock frequency itself, the flash memory has NOT
    // been configured for instruction fetches at a higher frequency. Omitting these two lines WILL
    // cause your board to attempt to fetch instructions before flash memory has properly fetched
    // them, meaning your board will begin to read garbage instructions and will no longer be
    // flashable without a factory reset.
    FlashCtl_setWaitState(FLASH_BANK0, 2);
    FlashCtl_setWaitState(FLASH_BANK1, 2);

    // Set the system clock frequency to user-specified frequency
    CS_setDCOFrequency(SYSTEM_CLOCK);

    // After DCO is set, configure all other clock signals to use a source from DCO.
    CS_initClockSignal(CS_MCLK  , CS_DCOCLK_SELECT , CS_CLOCK_DIVIDER_1);
    CS_initClockSignal(CS_HSMCLK, CS_DCOCLK_SELECT , CS_CLOCK_DIVIDER_1);
    CS_initClockSignal(CS_SMCLK , CS_DCOCLK_SELECT , CS_CLOCK_DIVIDER_1);
    CS_initClockSignal(CS_ACLK  , CS_REFOCLK_SELECT, CS_CLOCK_DIVIDER_1);
```

Step 2: Set up Wall Clock Hardware Timer

```
// Initialize the main hardware timer under which all other software timers are based. This
// should be a periodic timer with the maximum load value supported and a prescaler of 1 in
// order to minimize the frequency of interrupts while keeping a high timer resolution.
Timer32_initModule(TIMER32_0_BASE, TIMER32_PRESCALER_1, TIMER32_32BIT, TIMER32_PERIODIC_MODE);
Timer32_setCount(TIMER32_0_BASE, LOADVALUE);

// Starts the main reference hardware timer and enables an interrupt which counts rollovers
Timer32_startTimer(TIMER32_0_BASE, false);

// Enable interrupts again, after all system timing has been set up properly
Interrupt_enableMaster();
Interrupt_enableInterrupt(INT_T32_INT1);
}
```

Step 2: Comments

- 32-bit timer
- Timer 0
- Pre-scaler: P $= 1$
- LOADVALUE: `0xFFFFFFFF`
- Mode: Periodic
- One-shot: false

Step 3: Declaring the Rollover Variable (`timer.c`)

- Declare a static volatile variable to count the timer rollovers

```
/** The reference counter which tracks how many rollovers
have occurred. Used in timing SWTimers. */

static volatile uint64_t hwTimerRollovers = 0;
```

`static volatile uint64_t hwTimerRollovers`

- Limited visibility: the variable is only accessible within the file where it is declared
- Program lifetime: the variable exists throughout the entire execution of the program and retains its value
- No compiler optimization: the compiler will not optimize access to this variable making sure that its value is alway written to or read from its designated memory location and allowing for potential external modifications

Typical Use Cases

- Hardware registers
- Flags or counters in interrupt service routines
- Shared variable in multithreaded applications

Step 3: Interrupt Service Routine (ISR) (*timer.c*)

- Interrupt handler called when the timer reaches 0 (expires)

```
 /**
 * The ISR used to increment the total number of rollovers
 *  which have passed. When the TIMER32_0_BASE timer expires,
 *  this ISR is automatically called. DO NOT DIRECTLY INVOKE
 *  THIS FUNCTION FROM YOUR CODE, or you WILL destroy the
 *  accuracy of ALL software timers in your code.
 */

void T32_INT1_IRQHandler()
{
    hwTimerRollovers++;
    Timer32_clearInterruptFlag(TIMER32_0_BASE);
}
```

Step 4: Constructing the SWTimer Struct (Timer.h)

```
struct _SWTimer
{
    // The number of hardware timer cycles which must elapse
    // before the timer expires
    uint64_t cyclesToWait;

    // The starting counter value of the hardware timer, set
    // when the timer is started
    uint32_t startCounter;

    // The starting rollover value of the hardware timer, set
    // when the timer is started
    uint32_t startRollovers;
};
typedef struct _SWTimer SWTimer;
```

Step 4: Constructing the SWTimer Struct (Timer.c)

```
/**
 * Constructs a new Software Timer, using a wait time in milliseconds. The timer uses the
 * hwTimerRollovers variable to keep track of its reference time, and is based off of time pass
 * under the TIMER32_0_BASE. When first constructed, this timer is NOT conditioned to start. Be
 * any calls to SWTimer_expired(), SWTimer_elapsedTimeUS(), or SWTimer_percentElapsed(), you MU
 * FIRST CALL the SWTimer_start() method.
 *
 * @param waitTime_ms:  The amount of time this timer measures before expiration
 * @return a SWTimer object
 */
SWTimer SWTimer_construct(uint64_t waitTime_ms)
{
    SWTimer timer;

    timer.startCounter = 0;
    timer.startRollovers = 0;

    uint64_t counterClock = SYSTEM_CLOCK / PRESCALER;
    uint64_t cyclesPerMillisecond = counterClock / MS_DIVISION_FACTOR;
    timer.cyclesToWait = cyclesPerMillisecond * waitTime_ms;

    return timer;
}
```

Step 5: Start the Timer for the Application (Timer.c)

- *timer_p for the application
- startRollovers set to current number of rollovers, R1
- startCounter set to C1

```
/**
 * Starts a constructed timer by reading the current number
 * of rollovers and current load value in TIMER32_0_BASE.
 *
 * @param timer_p:    The SWTimer to start
 */
void SWTimer_start(SWTimer* timer_p)
{
    timer_p->startCounter = Timer32_getValue(TIMER32_0_BASE);
    timer_p->startRollovers = hwTimerRollovers;
}
```

Step 6: Compute Elapsed Cycles (`Timer.c`)

- Number of elapsed counter cycles $= (\mathrm{C1} - \mathrm{C2}) + (\mathrm{R2} - \mathrm{R1}) \times (N + 1)$
- Number of elapsed rollovers: $\mathrm{R2} - \mathrm{R1}$
- currentCounter: $\mathrm{C2}$

```
/**
 * A helper method to determine how many cycles have elapsed since the SWTimer started. As the
 * user,  * you most likely do NOT need to call this method outside of the Timer.c file. This
 * method is used  * in calculating how much time has elapsed for each of the methods below.
 * If the timer was never  * started, this function instead will return the number of cycles
 * from the start of the program's execution.
 *
 * @param timer_p:    The SWTimer with which we measure the number of cycles elapsed
 * @return the number of cycles elapsed since the timer started.
 */
uint64_t SWTimer_elapsedCycles(SWTimer* timer_p)
{
    uint64_t rollovers = hwTimerRollovers - timer_p->startRollovers;
    uint64_t startCounter = timer_p->startCounter;
    uint64_t currentCounter = Timer32_getValue(TIMER32_0_BASE);
    uint64_t elapsedCycles = (rollovers * (LOADVALUE + 1)) + startCounter - currentCounter;
    return elapsedCycles;
}
```

Step 7: Additional Timer Functionality (Timer.h)

```
// Determines if the timer has expired - i.e. if enough time
// has passed since the timer was started
bool SWTimer_expired(SWTimer* timer_p);

// Stopwatch functionality. Determines the number of
// microseconds since the timer was started.
uint64_t SWTimer_elapsedTimeUS(SWTimer* timer_p);

// Determines the percentage of time elapsed before
// expiration. When the timer is started, the value
// is 0.0, and as time elapses, the value rises to 1.0.
double SWTimer_percentElapsed(SWTimer* timer_p);
```

Step 7: Additional Timer Functionality (Timer.c)

- Is the timer expired?

```
/**
 * Determines whether the proper amount of time has elapsed
 * on this timer.
 *
 * @param timer_p:    The target timer used in determining
 * expiration
 * @return true if the timer is expired and false otherwise
 */
bool SWTimer_expired(SWTimer* timer_p)
{
    uint64_t elapsedCycles = SWTimer_elapsedCycles(timer_p);
    return elapsedCycles >= timer_p->cyclesToWait;
}
```

Step 7: Additional Timer Functionality (Timer.c)

```
/**
 * Returns the number of MICROSECONDS which have elapsed since the timer
 * was started. If the timer was never started, this function returns
 * the number of MICROSECONDS since the program started running. We compute
 * the time elapsed in microseconds and NOT in milliseconds because more
 * precise calculations can be made using microseconds.
 *
 * @param timer_p:  The target timer used in determining the elapsed time
 * @return the number of microseconds elapsed since the timer was started
 */
uint64_t SWTimer_elapsedTimeUS(SWTimer* timer_p)
{
    uint64_t elapsedCycles = SWTimer_elapsedCycles(timer_p);
    uint64_t counterClock = SYSTEM_CLOCK / PRESCALER;
    uint64_t cyclesPerMicrosecond = counterClock / US_DIVISION_FACTOR;
    uint64_t elapsedTime_us = elapsedCycles / cyclesPerMicrosecond;
    return elapsedTime_us;
}
```

Re-implementing Basic Timer with SWTimer

```
int main(void)
{
    initialize();     // initialize all the peripherals
    InitSystemTiming();  // initialize wall clock system
    static SWTimer myTimer;
    static bool startTimer = true;

    while (1) {
        if (!S1isPressed())  // If the button is not pressed, keep the LED off
            TurnOff_Launchpad_LED1();
        else
            TurnOn_Launchpad_LED1();
        if (startTimer){
            myTimer = SWTimer_construct(1000);
            SWTimer_start(&myTimer);
            startTimer = false;
        }
        else {
            if (SWTimer_expired(&myTimer)){
                Toggle_Launchpad_Green_LED2();
                startTimer = true;
            }
        }
    }
}
```

Summary

- Programmable hardware time modules
- Pre-scaler used to expand timer range
- Operation modes
  - One-shot versus periodic
- Software timers